

NFE204 - Bases de Données Documentaires et Distribuées

Projet : Intégration des Données pour une Analyse de Sentiments

Portés sur des Films

Etude : Etude du framework Apache Cassandra (Architecture,
Réplication/Distribution des Données)

Fabrice LEBEL

Email: fabrice.lebel@live.fr

Port.: 06.13.60.28.72

25 septembre 2015

Table des matières

Introduction	3
1 Intégration des Données pour une Analyse de Sentiments Portés sur des Films	4
1.1 Introduction	4
1.2 Modélisation Statique de l'Intégration des Données	4
1.3 Implantation et Résultats	5
1.4 Commentaires	8
2 Etude du Système NoSQL Apache Cassandra	8
2.1 Introduction	8
2.2 Architecture Distribuée	9
2.3 Gestion de la Réplication et Distribution des Données	9
Références	14
Annexes	15
A1. Classe Comment	15
A2. Classe Comments	16
A3. Classe DataBase	17
A4. Classe HBaseDB	18
A5. Classe CassandraDB	19
A6. Classe Driver	21
A7. Classe HBaseDriver	22
A8. Classe CassandraDriver	23

Introduction

Dans le cadre de ce projet nous présentons successivement

1. un **mini-projet** portant sur l'intégration des données nécessaires à la réalisation d'une étude concernant l'**analyse de sentiments, positifs ou négatifs, portant sur des films**. Elle consiste, schématiquement, à entraîner des modèles à distinguer les mots plutôt négatifs des mots plutôt positifs de façon à pouvoir évaluer automatiquement si un commentaire de film est positif ou négatif.
Pour ce faire, nous avons choisi d'expérimenter le stockage des données dans les bases NoSQL orientées colonnes
 - **Apache HBase**
 - **Apache Cassandra**.Ces données, bénéficiant d'un environnement distribué, pourront par la suite être utilisés par les logiciels/frameworks tels que R, Hadoop Mapreduce, Apache Spark, H2O, etc.
2. une **étude du système NoSQL Apache Cassandra** qui a été développée à l'origine par la société Yahoo! et qui nous semble en très forte progression au sein de la communauté 'Big Data'. Nous nous intéressons plus particulièrement aux aspects architecture et réplication/distribution des données au sein de ce framework.

Les **données** utilisées dans ce projet et celui du **module STA211** proviennent d'une étude réalisée par MAAS et *al.* (2011) (<http://ai.stanford.edu/~amaas/data/sentiment/>). Elles consistent en 25,000 **commentaires de films** très connotés négativement ou positivement pour la phase d'apprentissage ainsi que 25,000 commentaires pour la phase de test. Chaque film possède au maximum 30 commentaires. Ces commentaires ont été récoltés à partir du site Internet Movie Database (IMDb). Les notes des commentateurs vont de 1 à 10. L'échantillon de **commentaires négatifs** comprend les commentaires dont la **note** est **inférieure ou égale à 4/10**. L'échantillon de **commentaires positifs** contient les commentaires dont la **note** est **supérieure ou égale à 7/10**. Il est à noter que cette base de donnée contient aussi 50,000 commentaires sans étiquettes pour les méthodes d'apprentissage non-supervisées.

Dans le cadre de cette étude, les développements ont été réalisés dans l'environnement **GNU-Linux Opensuse** (version 13.2, <https://www.opensuse.org>), avec la distribution **Hadoop** (pour HDFS et HBase) de l'entreprise **Cloudera** installée **nativement** en mode 'standalone' (CDH-5.4.5, <http://www.cloudera.com>). Le **framework Cassandra** utilisé est celui du site Apache Cassandra (version 2.1.9, <http://cassandra.apache.org/>). Le langage de développement est **Oracle Java** (version 1.8.x, <http://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html>).

Pour la partie '**Integrated Development Environment**' (**IDE**) nous avons utilisé **IntelliJ IDEA** (version 14.1.2, <https://www.jetbrains.com/idea/>) et les librairies et dossiers contenant des librairies liés au projet sont les suivants :

- `org.apache.common.io:1.4.0`
- `/usr/lib/hadoop/`
- `/usr/lib/hadoop-hdfs/`
- `/usr/lib/hadoop-mapreduce/`
- `/usr/lib/hadoop-yarn/`
- `/usr/lib/hbase/`
- `/[install dir]/apache-cassandra-2.1.9/lib/`
- `/[install dir]/apache-cassandra-2.1.9/tools/lib/`
- `org.jsoup.jsoup:1.8.3` pour la suppression des tags HTML dans les commentaires pour lesquels nous avons rencontré des problèmes lors de l'intégration des données dans Cassandra.

1 Intégration des Données pour une Analyse de Sentiments Portés sur des Films

1.1 Introduction

Les **données** des commentaires sont sous la forme de **fichiers texte** contenant chacun un unique commentaire d'un individu. Ces fichiers sont répartis dans deux dossiers, **train** et **test**, qui eux même possèdent deux sous-dossiers, **pos** et **neg**, contenant respectivement les fichiers des commentaires positifs et négatifs.

Chaque commentaire de film possède un **identifiant** et une note (allant de 1 à 10) qui sont encodées dans le nom du fichier sous la forme : **id_note.txt**. Ainsi, le fichier **train/pos/200_8.txt** contient un commentaire positif appartenant à l'échantillon d'entraînement, ayant comme identifiant unique 200 et une note de 8 pour le film.

Dans la suite nous présentons

1. une **modélisation statique du problème** qui nous servira à **identifier** les **objets/classes principales** à l'élaboration du programme nous permettant d'intégrer les données dans des tables HBase et Cassandra.
2. l'**implantation** des **classes** en **Java** ainsi que les **résultats** de l'**intégration** des **données**
3. quelques **commentaires** sur le projet.

1.2 Modélisation Statique de l'Intégration des Données

Dans cette section nous présentons une partie de la **modélisation statique** qui nous permet d'intégrer les enregistrements représentant les commentaires portés sur des films.

Lors de notre modélisation, les **principaux objets identifiés** sont présentés dans la **Table 1**.

Objet (Classe potentielle)	Commentaire	Classe Parente
Comment	commentaire d'un film	
Comments	liste de commentaires de films	
DataBase	base de données	
HBase	base de données Apache HBase	DataBase
Cassandra	base de données Apache Cassandra	DataBase
Driver	classe 'virtuelle' gérant l'intégration des données	
HBaseDriver	classe gérant l'intégration des données dans HBase	Driver
CassandraDriver	classe gérant l'intégration des données dans Cassandra	Driver

TABLE 1 – Liste des objets/classes identifiés.

Nous présentons ci-après les **attributs** de la **classe 'Comment'** qui est une des classes centrales de notre modèle (cf. **Table 2**). Nous utilisons aussi ces attributs pour les **colonnes des tables** des base de données. Le champ **commentId** est défini comme étant la **clé primaire**. Notons que ce champ est nécessaire car deux fichiers de commentaires se trouvant respectivement dans les dossiers **train/pos/** et **train/neg/** peuvent avoir le même identifiant de fichiers.

Attribut	Type	Commentaire
commentID	int	Identifiant unique d'un commentaire dans la table (clé primaire).
fileID	String	Identifiant unique du fichier dans son dossier pos ou neg .
commentRate	String	Note du film.
comment	String	Le commentaire porté sur le film.
commentType	String	Commentaire positif ('pos') ou négatif ('neg').

TABLE 2 – Attributs de base de la classe 'Comment'.

La **Figure 1** présente l'architecture des relations entre les différents objets du modèle.

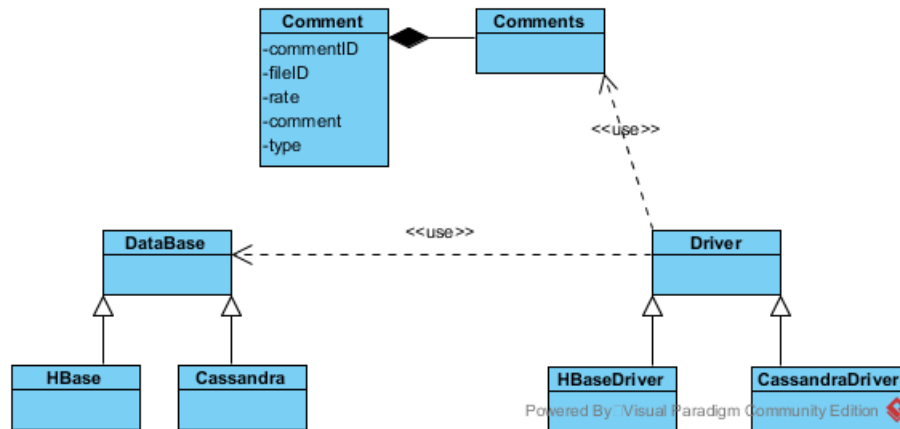


FIGURE 1 – Diagramme des classes du modèle

1.3 Implantation et Résultats

De la modélisation statique de la section précédente nous avons implanté les différentes classes dont le **code Java** est présenté en Annexes.

La classe **HBaseDriver** permet d'intégrer les données dans deux tables HBase **train** et **test**. Pour ce faire, un driver Java pour HBase est utilisé (<http://hbase.apache.org/book.html>). La documentation de l'API Java HBase se trouve à l'adresse <http://hbase.apache.org/apidocs/index.html>.

Les **Figure 2** et **Figure 3** présentent respectivement la structure interne des tables **train** et **test**, dans HBase et le résultat d'une requête de sélection des données.

```

hbase(main):049:0* describe 'train'
Table train is ENABLED
train
COLUMN FAMILIES DESCRIPTION
{NAME => 'data', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false',
KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING
=> 'NONE', TTL => 'FOREVER', COMPRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true',
BLOCKSIZE => '65536', REPLICATION_
SCOPE => '0'}
1 row(s) in 0.0320 seconds

hbase(main):050:0> describe 'test'
Table test is ENABLED
test
COLUMN FAMILIES DESCRIPTION
{NAME => 'data', BLOOMFILTER => 'ROW', VERSIONS => '1', IN_MEMORY => 'false',
KEEP_DELETED_CELLS => 'FALSE', DATA_BLOCK_ENCODING
=> 'NONE', TTL => 'FOREVER', COMPRESSION => 'NONE', MIN_VERSIONS => '0', BLOCKCACHE => 'true',
BLOCKSIZE => '65536', REPLICATION_
SCOPE => '0'}
1 row(s) in 0.0310 seconds

hbase(main):051:0>
  
```

FIGURE 2 – Structure interne des tables **train** et **test** dans HBase.

```

hbase(main):064:0> scan 'train',{LIMIT=>2}
ROW                                COLUMN+CELL
\x00\x00\x00\x00                 column=data:comment, timestamp=1443059373824, value=Zentropa has much in common with The Third Man, another noir-like film set among the rubble of postwar Europe. Like TTM, Zentropa is a well-crafted, intimate camera work. There is an innocent American who gets emotionally involved in the story, but he doesn't really understand, and whose naivety is all the more striking in contrast to the other characters. <br /><br />But I'd have to say that The Third Man has a more well-crafted story. Zentropa is a bit disjointed in this respect. Perhaps this is intentional: it is a film about a man's nightmare, and making it too coherent would spoil the effect. <br /><br />The film is a relentlessly grim--"noir" in more than one sense; one never sees the sun shine. Grief-stricken, and frightening.
\x00\x00\x00\x00                 column=data:fileid, timestamp=1443059373824, value=127
\x00\x00\x00\x00                 column=data:rate, timestamp=1443059373824, value=7
\x00\x00\x00\x00                 column=data:type, timestamp=1443059373824, value=pos
\x00\x00\x00\x01                 column=data:comment, timestamp=1443059373850, value=Zentropa is the most original film I've seen in years. If you like unique thrillers that are influenced by film noir, this is the right cure for all of those Hollywood summer blockbusters clogging the theaters. Zentropa is the best work. It is flashy without being distracting and offers the perfect combination of suspense and dark humor. It's too bad he decided handheld cameras were the wave of the future. Hard to say who talked him away from the style he exhibits here, but it's everyman's fault. He went into his heavily theoretical dogma direction instead.
\x00\x00\x00\x01                 column=data:fileid, timestamp=1443059373850, value=126
\x00\x00\x00\x01                 column=data:rate, timestamp=1443059373850, value=10
\x00\x00\x00\x01                 column=data:type, timestamp=1443059373850, value=pos
2 row(s) in 0.0220 seconds

hbase(main):065:0>

```

FIGURE 3 – Exemple d'enregistrements de la table `train` dans HBase.

Le programme d'intégration des données dans Cassandra utilise le driver 'Datastax Java Driver 2.0 for Apache Cassandra' (<http://docs.datastax.com/en/developer/java-driver/2.0/java-driver/whatsNew2.0.html>). La documentation de l'API se trouve à l'adresse <http://docs.datastax.com/en/drivers/java/2.0/>.

Dans Apache Cassandra nous avons créé la base de donnée `nfe204`. Celle-ci contient deux tables `train` et `test` contenant respectivement les commentaires de l'échantillon d'entraînement et ceux de test.

Les **Figure 4** et **Figure 5** présentent respectivement la structure interne des tables, `train` et `test`, dans Cassandra et le résultat d'une requête de sélection des données.

```

fabrice@fab-packard:~/bin/apache-cassandra-2.1.9> bin/cqlsh
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 2.1.9 | CQL spec 3.2.0 | Native protocol v3]
Use HELP for help.
cqlsh> DESC nfe204;

CREATE KEYSPACE nfe204 WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'}
AND durable_writes = true;

CREATE TABLE nfe204.test (
    comment_id int PRIMARY KEY,
    comment text,
    file_id text,
    rate int,
    type text
) WITH bloom_filter_fp_chance = 0.01
    AND caching = '{"keys":"ALL", "rows_per_partition":"NONE"}'
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy'}
    AND compression = {'sstable_compression': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND dclocal_read_repair_chance = 0.1
    AND default_time_to_live = 0
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair_chance = 0.0
    AND speculative_retry = '99.0PERCENTILE';

CREATE TABLE nfe204.train (
    comment_id int PRIMARY KEY,
    comment text,
    file_id text,
    rate int,
    type text
) WITH bloom_filter_fp_chance = 0.01
    AND caching = '{"keys":"ALL", "rows_per_partition":"NONE"}'
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy'}
    AND compression = {'sstable_compression': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND dclocal_read_repair_chance = 0.1
    AND default_time_to_live = 0
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair_chance = 0.0
    AND speculative_retry = '99.0PERCENTILE';

cqlsh>

```

FIGURE 4 – Structure interne des tables `train` et `test` dans Cassandra.

```

cqlsh:nfe204>
cqlsh:nfe204>
cqlsh:nfe204> SELECT * FROM train LIMIT 2;

comment_id | comment

-----+-----
| file_id | rate | type
-----+-----
4317 | This film plays in the 60s and is about an Italian family: Romano, his wife Rosa and their two children Gigi and Giancarlo emigrate from Solino in Italy to Duisburg in the Ruhr area. I like this film, because I think it is quite realistic: it shows problems which many foreign families have when they come to another country: they have to get used to a new culture, a new environment and this can be difficult: especially if you don't know the language.... It is difficult for the family but they find a way: they open a restaurant which offers typical Italian food, and it is named "Solino", like their hometown. The film also shows different conflicts - Gigi and Giancarlo fall in love with the same girl, and although Rosa has to work very hard, Romano refuses to pay money to engage more workers, etc. etc. But stop, I don't want to tell you how it goes on. You should watch the film yourself, it's a nice one - I have also made a Referat about it and examined scenes which show different cultural attitudes. And there are a few... | 4258 | 9 | pos
3372 |
This is easily a 9. Michel Serrault, known more for comic roles in the earlier part of his acting career, does a stunning, even worryingly stunning job of impersonating Dr Petiot, a legendary French serial killer. He is just so believable at every and any moment in the film, that the actor completely disappears behind the character - only the very best ever achieve this feat, and when they do it is only in a handful of parts at best. The whole story (a real story which happened in 20th century France) is so powerful, so sinister - it makes for a very strong film that one remembers for a long, long time. | 3411 | 9 | pos
(2 rows)
cqlsh:nfe204>

```

FIGURE 5 – Exemple d’enregistrements de la table `train` dans la base `nfe204` de Cassandra.

1.4 Commentaires

Pour conclure notre propos, nous pouvons faire les remarques suivantes :

- L’intégration des données nous a permis d’identifier des **commentaires contenant des tags HTML** et de les supprimer permettant ainsi un premier nettoyage.
- Nous aurions pu intégrer l’ensemble des **données d’entraînement** et de test **dans une seule table** en ajoutant par exemple une colonne `dataset` qui prendrait les valeurs `train` ou `test`.
- La **prise en main** du framework **Apache Cassandra** se **approche** de celle des **bases de données relationnelles** type MySQL ou Oracle grâce à ses outils disponibles et son langage d’interrogation CQL. A notre avis, le shell HBase est un peu moins convivial.

2 Etude du Système NoSQL Apache Cassandra

2.1 Introduction

Apache Cassandra est une base de données non-relationnelle créée initialement par la société Yahoo! et passée dans le domaine libre, sous licence Apache, en 2008.

C’est un moteur de **base de données orientée colonnes**. Dans ce type de représentation, les données sont regroupées par **familles de colonnes** (**‘column families’**) qui peuvent ne pas avoir le même nombre de colonnes. De ce fait, le schéma de la base de données n’est pas fixé à l’avance.

Chaque colonne est définie par un nom (chaîne de caractères, entier, UUID...) et **contient**

- **une valeur ou un ensemble de colonnes** (appelé aussi sous-famille de colonnes)
- un **‘timestamp’** ou horodatage.

Outre la notion de familles de colonnes, Apache Cassandra manipule des objets proches de ceux que

l'on trouve dans les base de données relationnelles :

- **Keyspace** : c'est un conteneur pour les tables de données et les index. Nous pouvons représenter un keyspace comme une base de donnée, conteneur de tables (relations).
- **Table** : c'est une table analogue à celles des bases de données relationnelles
- **Clé-primaire ('primary key')** : permet d'identifier de manière unique une ligne d'une table sur les différents nœuds du cluster
- **Index** : comme dans les bases de donnée relationnelles il permet d'augmenter les performance de lecture des enregistrements d'une table.

Dans cette section nous nous intéressons plus particulièrement à

- l'**architecture distribuée** du framework Apache Cassandra
- la **distribution** et la **réplication** des données au sein d'un cluster Apache Cassandra.

Notons que notre présentation se base principalement sur les ouvrages de BRUCHEZ (2013)... Nous renvoyons le lecteur à ces ouvrages pour de plus amples informations.

2.2 Architecture Distribuée

Apache Cassandra a une **architecture 'peer-to-peer' c.-à-d. sans notion de serveur/nœud maître-esclave** (ou encore '**masterless**') et utilisant une **topologie en anneau** appelée aussi **topologie Chord**. Dans ce type d'architecture chaque nœud joue le même rôle et ils communiquent entre eux grâce à un protocole nommé **protocole de bavardage** ou '**gossip protocol**'. Ce protocole permet aux nœuds du cluster d'échanger des informations concernant leur état, l'ajout/retrait d'un nœud, la mise à jour des données...

En ce qui concerne l'aspect de **mise à l'échelle** ou '**scalability**', elle est gérée en **ajoutant, 'à chaud'** (c.-à-d. sans arrêt/redémarrage de la grappe de serveurs), de **nouveaux nœuds**.

2.3 Gestion de la Réplication et Distribution des Données

Dans un système NoSQL les données doivent être distribuées équitablement entre les différents nœuds composant la grappe de serveurs. Comme l'indique BRUCHEZ (2013), p. 69 et sv., une **approche simple** peut consister à **générer une clé de hachage pour chaque nœud** à partir des clés primaire des données, du nombre de nœuds dans le cluster et de l'**opérateur modulo**. Le résultat d'un hachage correspond donc à l'identité d'un nœud dans le cluster. Mais cette approche simple à l'inconvénient d'entraîner des migrations des données lorsque des nœuds sont ajoutés ou retirés au cluster. La **Table 3** illustre cette problématique de migration des données avec deux clusters, un composé initialement de 4 nœuds puis de 5 nœuds.

Clé primaire	Cluster 4 nœuds	Cluster 5 nœuds	Migration des données
45	$45\%4 = 1 \equiv N1$	$45\%5 = 0 \equiv N0$	$N1 \rightarrow N0$
46	$46\%4 = 2 \equiv N2$	$46\%5 = 1 \equiv N1$	$N2 \rightarrow N1$
47	$47\%4 = 3 \equiv N3$	$47\%5 = 2 \equiv N2$	$N3 \rightarrow N2$
48	$48\%4 = 0 \equiv N0$	$48\%5 = 3 \equiv N3$	$N0 \rightarrow N3$
49	$49\%4 = 1 \equiv N1$	$49\%5 = 4 \equiv N4$	$N1 \rightarrow N4$
50	$50\%4 = 2 \equiv N2$	$50\%5 = 0 \equiv N0$	$N2 \rightarrow N0$

TABLE 3 – Illustration de la migration des données lors de l'ajout d'un nœud à un cluster avec un algorithme simple de hachage de la clé primaire des données. Le résultat du modulo (%) indique l'identité du nœud $Ni, i = 0..4$, où la donnée sera stockée.

Afin d'éviter cette migration massive de données entre les nœuds lors de l'ajout ou du retrait de nœuds, un algorithme plus sophistiqué appelé **hachage cohérent** ou encore '**consistent hashing**' est utilisé.

L'**idée de base** du **hachage cohérent** est que l'**application** d'une même **fonction de hachage** aux **identifiants** des **nœuds** du **cluster** (généralement à partir des adresses IP combinées au port de

communication) et aux **clés primaires** des **lignes** des tables de données **gène**re des **valeurs** dans le **même espace d'identifiants**. Ces valeurs de hachage sont ensuite placées sur un cercle.

Afin d'illustrer notre propos, **supposons** que les **valeurs de hachage** sont **encodées sur m bits**. Nous disposons alors de 2^m points. Chaque identifiant de nœud ou de clé primaire peut alors être choisi parmi ces points. Si $m = 7$ nous disposons de $2^7 = 128$ points (entiers) compris entre 0 et 127. Si nous les plaçons sur un cercle nous avons la représentation de la **Figure 6**.

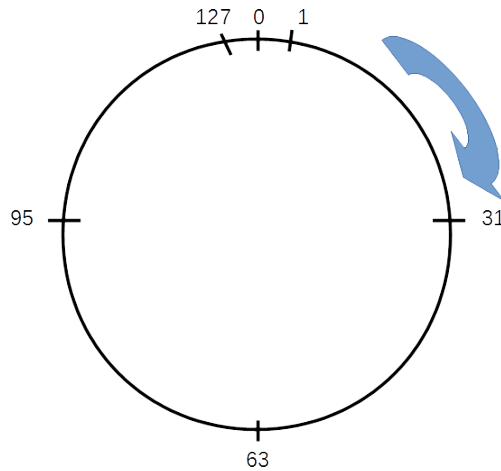


FIGURE 6 – Représentation sur un cercle des valeurs de hachage encodées sur $m = 7$ bits, soit $2^7 = 128$ valeurs comprises entre 0 et 127.

En ce qui concerne le **rou**tage des **messages** dans un **système Chord**, **chaque nœud connaît ses successeurs** (nœuds situés dans le sens des aiguilles d'une montre) et **parfois ses prédécesseurs** (nœuds situés en sens inverse des aiguilles d'une montre). L'algorithme le plus utilisé est celui des **doigts de la main** ou '**finger algorithm**'. Dans un système Chord, chaque nœud maintient une 'finger table' dont les entrées sont calculées par la formule décrite ci-après.

Posons

- m = nombres de bits pour l'encodage des valeurs de hachage
- i = i -ème entrée de la 'finger table', $i = 0, 1, 2, \dots, m - 1$
- j = indice d'un nœud
- $ft(i)$ = valeur dans la 'finger table' de l'entrée i .

Nous avons alors

$$(1) \quad ft(i) = (j + 2^i) \mod 2^m.$$

Posons

- N_j = identifiant du nœud j
- $HashFunc(x)$ = fonction de hachage appliquée à la clé x .

et supposons que nous disposons de 4 nœuds ('servent') dont les valeurs de hachage, encodées sur $m = 7$ bits, sont indiquées dans la **Table 4**.

Clé	Identifiant sur le cercle
N_0	$HashFunc(N_0) \mod 2^7 = 0$
N_1	$HashFunc(N_1) \mod 2^7 = 31$
N_2	$HashFunc(N_2) \mod 2^7 = 63$
N_3	$HashFunc(N_3) \mod 2^7 = 95$

TABLE 4 – Valeurs des identifiants des nœuds d'un cluster (N_j) encodées sur $m = 7$ bits.

La 'finger table' du nœud $N1$ est représentée dans la **Table 5**.

Entrée i	$ft(i)$	Successeur de $ft(i)$
0	$(31 + 2^0) \bmod 2^7 = 32$	$63 \equiv N2$
1	$(31 + 2^1) \bmod 2^7 = 33$	$63 \equiv N2$
2	$(31 + 2^2) \bmod 2^7 = 35$	$63 \equiv N2$
3	$(31 + 2^3) \bmod 2^7 = 39$	$63 \equiv N2$
4	$(31 + 2^4) \bmod 2^7 = 47$	$63 \equiv N2$
5	$(31 + 2^4) \bmod 2^7 = 63$	$95 \equiv N3$
6	$(31 + 2^6) \bmod 2^7 = 95$	$0 \equiv N0$

TABLE 5 – 'Finger table' du nœud $N1$ (ID = 31).

Donnons maintenant un exemple d'affectation des lignes de données aux nœuds d'un cluster. Pour ce faire, nous posons

$$Ki = \text{clé primaire de la ligne de donnée } i.$$

Les **valeurs de hachage** pour les clés primaires des lignes de données et les identifiants de nœuds regroupées dans la **Table 6**. Ces **valeurs** sont **comprises entre 0 et 127**. Nous débutons par un cluster composé de 4 nœuds ($N0, \dots, N3$) puis nous analysons l'impact de l'introduction d'un cinquième nœud ($N4$).

Clé	Identifiant sur le cercle
$N0$	$HashFunc(N0) \bmod 2^7 = 0$
$K0$	$HashFunc(K0) \bmod 2^7 = 20$
$K1$	$HashFunc(K1) \bmod 2^7 = 30$
$N1$	$HashFunc(N1) \bmod 2^7 = 31$
$K2$	$HashFunc(K2) \bmod 2^7 = 50$
$N2$	$HashFunc(N2) \bmod 2^7 = 63$
$K3$	$HashFunc(K3) \bmod 2^7 = 70$
$K4$	$HashFunc(K4) \bmod 2^7 = 85$
$K5$	$HashFunc(K5) \bmod 2^7 = 90$
$N3$	$HashFunc(N3) \bmod 2^7 = 95$
$K6$	$HashFunc(K6) \bmod 2^7 = 100$
$K7$	$HashFunc(K7) \bmod 2^7 = 110$
$N4$	$HashFunc(N4) \bmod 2^7 = 80$

TABLE 6 – Valeurs de hachage des clés primaires des lignes de données (Ki) et des identifiants des nœuds d'un cluster (Nj).

Si nous posons les identifiants des nœuds et des clés primaires sur un anneau et que nous utilisons l'**algorithme d'affectation des identifiants des lignes de données** suivant : ??? nous obtenons la **Figure 7**.

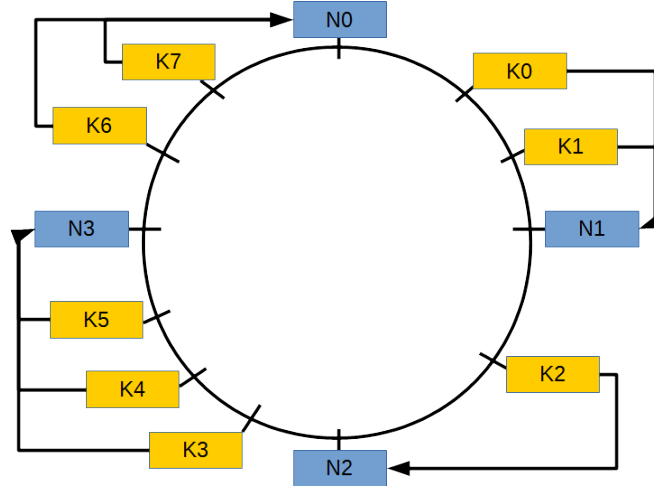


FIGURE 7 – Affectation des lignes de données dans un cluster de 4 nœuds dans une topologie 'peer-to-peer' en anneau.

Si nous **introduisons un nouveau nœud** $N4$ d'ID 80 la nouvelle 'finger table' du nœud $N1$ est décrite dans la **Table 7**.

Entrée i	$ft(i)$	Successeur de $ft(i)$
0	$(31 + 2^0) \bmod 2^7 = 32$	$63 \equiv N2$
1	$(31 + 2^1) \bmod 2^7 = 33$	$63 \equiv N2$
2	$(31 + 2^2) \bmod 2^7 = 35$	$63 \equiv N2$
3	$(31 + 2^3) \bmod 2^7 = 39$	$63 \equiv N2$
4	$(31 + 2^4) \bmod 2^7 = 47$	$63 \equiv N2$
5	$(31 + 2^5) \bmod 2^7 = 63$	$80 \equiv N4$
6	$(31 + 2^6) \bmod 2^7 = 95$	$0 \equiv N0$

TABLE 7 – 'Finger table' du nœud $N1$ (ID = 31) après introduction du nœud $N4$ d'ID 80.

La nouvelle affectation des lignes de données représentée dans la **Figure 8**.

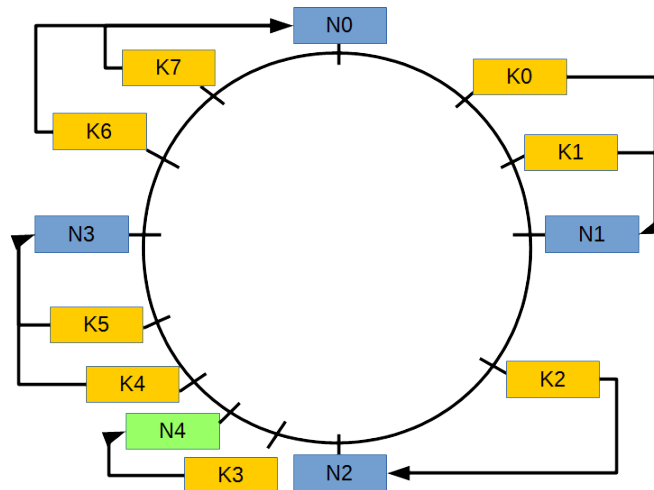


FIGURE 8 – Ré-affectation des lignes de données dans un cluster de 5 nœuds dans une topologie 'peer-to-peer' en anneau après ajout du nœud $N4$.

Si maintenant nous supprimons un nœud du cluster initial afin d'obtenir une **topologie comprenant 3 nœuds**, nous obtenons la **Figure 9**.

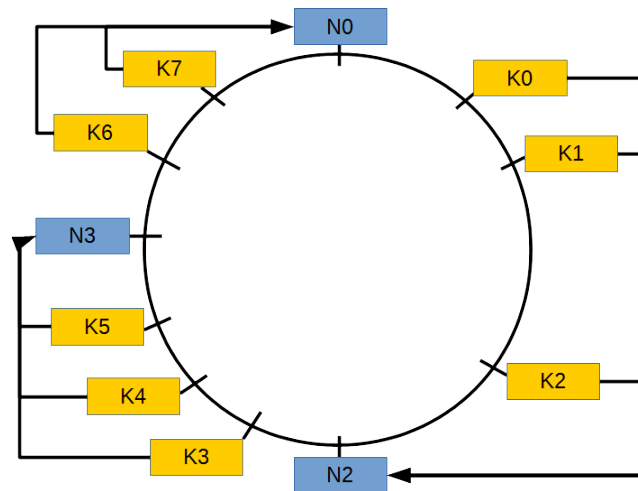


FIGURE 9 – Ré-affectation des lignes de données dans un cluster de 3 nœuds dans une topologie 'peer-to-peer' en anneau après suppression du nœud *N1*.

Dans Apache Cassandra, la **clé de hachage des nœuds** ou '**peer pointer**' est appelée **clé de partitionnement**. Les données sont quant-à elles distribuées grâce à un '**partitioner**' qui décide sur quel nœud certaines données doivent être distribuées en fonction de la valeur de leurs clés de hachage respectives.

Un autre paramètre d'Apache Cassandra est le **facteur de réplication** ('**replication factor**') qui détermine le nombre de nœuds disposant d'une copie (ou **réplicat**) de la même ligne d'une table. A titre d'exemple, un facteur de réplication de 2 indique que 2 nœuds du 'cluster' contiendront une copie d'une même ligne.

Références

- AMINI, M.-R., GAUSSIÉ, E. (2013). *Recherche d'information (Applications, modèles et algorithmes - Fouille de données, décisionnel et big data)*. Eyrolles, Paris.
- BRUCHEZ, R. (2013). *Les bases de données NoSQL (Comprendre et mettre en oeuvre)*. Eyrolles, Paris.
- MAAS, A. L., DALY, R. E., PHAM, P. T., HUANG, D., NG, A. Y., POTTS, C. (2011). « Learning Word Vectors for Sentiment Analysis ». In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics : Human Language Technologies*, pages 142–150, Portland, Oregon, USA. Association for Computational Linguistics.
- RIGAUX, P. (2013). *Bases de données documentaires et distribuées*. Creative Commons Attribution.

A1. Classe Comment

```
1 import org.jsoup.Jsoup;
2
3 /**
4  * Created by fabrice on 9/24/15.
5  */
6 public class Comment {
7     private int commentIndex;
8     private String fileId;
9     private String commentRate;
10    private String comment;
11    private String commentType;
12
13    public Comment(int commentIndex, String fileId, String rate, String comment,
14        String commentType) {
15        this.commentIndex = commentIndex;
16        this.fileId = fileId;
17        this.commentRate = rate;
18        this.comment = comment;
19        this.commentType = commentType;
20    }
21
22    public Integer getCommentIndex() {
23        return this.commentIndex;
24    }
25
26    public String getCommentRate() {
27        return this.commentRate;
28    }
29
30    public String getComment() {
31        return Jsoup.parse(this.comment).text();
32    }
33
34    public String getFileID() {
35        return this.fileId;
36    }
37
38    public String getCommentType() {
39        return this.commentType;
40    }
41
42    @Override
43    public String toString() {
44        String result = "{" + this.commentIndex + ";" + this.fileId + ";"
45            + this.commentRate + ";" + this.comment + ";" + this.commentType + "}";
46        return result;
47    }
48 }
```

A2. Classe Comments

```
1 import java.io.File;
2 import java.io.IOException;
3 import java.nio.file.Files;
4 import java.nio.file.Paths;
5 import java.util.ArrayList;
6 import java.util.List;
7
8 import org.apache.commons.io.FilenameUtils;
9
10 /**
11  * Created by fabrice on 9/24/15.
12  */
13 public class Comments {
14     private String dataPath;
15     private ArrayList<Comment> comments;
16     private int lastCommentIndex;
17
18     public Comments(String path) {
19         this.dataPath = path;
20         this.comments = new ArrayList<>();
21         this.lastCommentIndex = -1;
22     }
23
24     public void setDataPath(String path) {
25         this.dataPath = path;
26     }
27
28     public ArrayList<Comment> getComments() {
29         return this.comments;
30     }
31
32     private String getCommentFromFile(String path) throws IOException {
33         String comment = "";
34
35         List<String> lines = Files.readAllLines(Paths.get(path));
36         for (String line: lines) {
37             comment += "\n" + line;
38         }
39         return comment.trim();
40     }
41
42     public void loadCommentFiles(String commentType) throws IOException {
43         File dir = new File(this.dataPath);
44         String[] files = dir.list();
45         if (files == null) {
46             System.out.println("Directory does not exist or is not a directory");
47         } else {
48             for (int i = 0; i < files.length; i++) {
49                 String[] fields = FilenameUtils.removeExtension(files[i]).split("_");
50                 if (fields.length > 0) {
51                     lastCommentIndex++;
52                     String value = this.getCommentFromFile(dataPath + files[i]);
53                     Comment comment = new Comment(this.lastCommentIndex, fields[0],
54                     fields[1], value, commentType);
55                     comments.add(comment);
56                 }
57             }
58         }
59     }
60 }
```


A3. Classe DataBase

```
1  /**
2   * Created by fabrice on 9/24/15.
3   */
4  public class DataBase {
5      public static final String IMDB_COLUMN_FAMILY = "imdb";
6      public static final String IMDB_COLUMN_INDEX = "comment_id";
7      public static final String IMDB_COLUMN_FILEID = "file_id";
8      public static final String IMDB_COLUMN_RATE = "rate";
9      public static final String IMDB_COLUMN_COMMENT = "comment";
10     public static final String IMDB_COLUMN_TYPE = "type";
11     //public static final String CASSANDRA_IP_ADDRESS = "127.0.0.1";
12
13     private String databaseName;
14
15     public DataBase(String databaseName) {
16         this.databaseName = databaseName;
17     }
18
19     public void setDataBaseName(String databaseName) {
20         this.databaseName = databaseName;
21     }
22
23     public String getDatabaseName() {
24         return this.databaseName;
25     }
26 }
```

A4. Classe HBaseDB

```
1 import org.apache.hadoop.conf.Configuration;
2 import org.apache.hadoop.hbase.HBaseConfiguration;
3 import org.apache.hadoop.hbase.HColumnDescriptor;
4 import org.apache.hadoop.hbase.HTableDescriptor;
5 import org.apache.hadoop.hbase.TableName;
6 import org.apache.hadoop.hbase.client.HBaseAdmin;
7 import org.apache.hadoop.hbase.client.HTable;
8 import org.apache.hadoop.hbase.client.Put;
9 import org.apache.hadoop.hbase.util.Bytes;
10
11 import java.io.IOException;
12
13 /**
14  * Created by fabrice on 9/24/15.
15  */
16 public class HBaseDB extends DataBase {
17     private Configuration configuration;
18     private HBaseAdmin hBaseAdmin;
19     private HTableDescriptor hTableDescriptor;
20
21     public HBaseDB(String dbName) throws IOException {
22         super(dbName);
23         this.configuration = HBaseConfiguration.create();
24         this.hBaseAdmin = new HBaseAdmin(this.configuration);
25         this.hTableDescriptor = new
26             HTableDescriptor(TableName.valueOf(super.getDatabaseName()));
27     }
28
29     public void createDataBase(Comments comments) throws IOException {
30         System.out.println("INFO: _Start_creation_of_HBase_database_"
31             + super.getDatabaseName() + " '...'");
32
33         this.hTableDescriptor.addFamily(new HColumnDescriptor(DataBase.IMDB_COLUMN_FAMILY));
34         this.hBaseAdmin.createTable(this.hTableDescriptor);
35         HTable hTable = new HTable(this.configuration, super.getDatabaseName());
36         for (Comment comment : comments.getComments()) {
37             Put p = new Put(Bytes.toBytes(comment.getCommentIndex()));
38             p.add(Bytes.toBytes(DataBase.IMDB_COLUMN_FAMILY),
39                 Bytes.toBytes(DataBase.IMDB_COLUMN_FILEID),
40                 Bytes.toBytes(comment.getFileID()));
41             p.add(Bytes.toBytes(DataBase.IMDB_COLUMN_FAMILY),
42                 Bytes.toBytes(DataBase.IMDB_COLUMN_RATE),
43                 Bytes.toBytes(comment.getCommentRate()));
44             p.add(Bytes.toBytes(DataBase.IMDB_COLUMN_FAMILY),
45                 Bytes.toBytes(DataBase.IMDB_COLUMN_COMMENT),
46                 Bytes.toBytes(comment.getComment()));
47             p.add(Bytes.toBytes(DataBase.IMDB_COLUMN_FAMILY),
48                 Bytes.toBytes(DataBase.IMDB_COLUMN_TYPE),
49                 Bytes.toBytes(comment.getCommentType()));
50             hTable.put(p);
51         }
52         hTable.close();
53
54         System.out.println("INFO: _HBase_database_" + super.getDatabaseName() + "'_created.");
55     }
56 }
```

A5. Classe CassandraDB

```
1 import com.datastax.driver.core.Cluster;
2 import com.datastax.driver.core.Session;
3 import org.jsoup.Jsoup;
4
5 /**
6  * Created by fabrice on 9/24/15.
7  */
8 public class CassandraDB extends DataBase {
9     private String clusterIPAddress;
10    private Cluster cluster;
11    private Session session;
12
13    public CassandraDB(String dbName, String clusterIPAddress) {
14        super(dbName);
15        this.clusterIPAddress = clusterIPAddress;
16    }
17
18    public void connect() {
19        // Connexion au cluster Cassandra
20        this.cluster = Cluster.builder().addContactPoint(this.clusterIPAddress).build();
21        // Creation d'une session
22        this.session = this.cluster.connect();
23    }
24
25    public void disconnect() {
26        // Deconnexion du cluster Cassandra
27        this.cluster.close();
28    }
29
30    public void createDataBase() {
31        System.out.println("INFO: _Start_creation_of_Cassandra_database_");
32        + super.getDatabaseName() + "...");
33        // Creation d'un keyspace avec une requete CQL
34        String query = "CREATE_KEYSPACE_" + super.getDatabaseName()
35            + "_WITH_replication_={ 'class ': 'SimpleStrategy ', 'replication_factor ':1}";
36        this.session.execute(query);
37        System.out.println("INFO: _Cassandra_database_ " + super.getDatabaseName()
38            + "...created.");
39    }
40
41    public void createIMDBTable(String tableName) {
42        System.out.println("INFO: _Start_creation_of_table_ " + super.getDatabaseName()
43            + "." + tableName + "...");
44        // Utilisation du keyspace
45        this.session.execute("USE_" + super.getDatabaseName());
46        // Creation de la table
47        String query = "CREATE_TABLE_" + tableName + "("
48            + DataBase.IMDB_COLUMN_INDEX + "_int_PRIMARY_KEY,"
49            + DataBase.IMDB_COLUMN_FILEID + "_text,"
50            + DataBase.IMDB_COLUMN_RATE + "_int,"
51            + DataBase.IMDB_COLUMN_COMMENT + "_text,"
52            + DataBase.IMDB_COLUMN_TYPE + "_text)";
53        this.session.execute(query);
54        System.out.println("INFO: _Table_ " + super.getDatabaseName() + "." + tableName
55            + "...created.");
56    }
57
58    public void insertIMDBRecords(Comments comments, String tableName) {
59        System.out.println("INFO: _Start_insertion_of_records_into_ "
60            + super.getDatabaseName() + "." + tableName + "...");
61        // Utilisation du keyspace
62        this.session.execute("USE_" + super.getDatabaseName());
63        // Insertion des enregistrements
64        for (Comment comment : comments.getComments()) {
```

```

65         String query = "INSERT INTO_" + tableName + "_("
66             + DataBase.IMDB_COLUMN_INDEX + ","
67             + DataBase.IMDB_COLUMN_FILEID + ","
68             + DataBase.IMDB_COLUMN_RATE + ","
69             + DataBase.IMDB_COLUMN_COMMENT + ","
70             + DataBase.IMDB_COLUMN_TYPE + ")_"
71             + "VALUES_"
72             + comment.getCommentIndex() + ","
73             + "'" + comment.getFileID() + "'" + ","
74             + comment.getCommentRate() + ","
75             + "'" + comment.getComment().replaceAll("'", "'') + "'" + ","
76             + "'" + comment.getCommentType() + "'" + "';";
77         //System.out.println(query);
78         this.session.execute(query);
79     }
80     System.out.println("INFO:_" + Records.inserted_into_ + " + super.getDatabaseName()
81         + "." + tableName + ".");
82 }
83 }

```

A6. Classe Driver

```
1  /**
2   * Created by fabrice on 9/24/15.
3   */
4  public class Driver {
5      public static final String DATASET_TRAIN_POS_PATH =
6          "/home/fabrice/Formations/CNAM/NFE204/aclImdb/train/pos/";
7      public static final String DATASET_TRAIN_NEG_PATH =
8          "/home/fabrice/Formations/CNAM/NFE204/aclImdb/train/neg/";
9      public static final String DATASET_TEST_POS_PATH =
10         "/home/fabrice/Formations/CNAM/NFE204/aclImdb/test/pos/";
11     public static final String DATASET_TEST_NEG_PATH =
12         "/home/fabrice/Formations/CNAM/NFE204/aclImdb/test/neg/";
13     public static final String POS_TAG = "pos";
14     public static final String NEG_TAG = "neg";
15 }
```

A7. Classe HBaseDriver

```
1 import java.io.IOException;
2
3 /**
4  * Created by fabrice on 9/24/15.
5  */
6 public class HBaseDriver extends Driver {
7     public static void main(String[] args) throws IOException {
8         // Chargement des commentaires d'entrainement positifs et negatifs
9         // a partir des fichiers
10        Comments trainComments = new Comments(Driver.DATASET_TRAIN_POS_PATH);
11        trainComments.loadCommentFiles(Driver.POS_TAG);
12        trainComments.setDataPath(Driver.DATASET_TRAIN_NEG_PATH);
13        trainComments.loadCommentFiles(Driver.NEG_TAG);
14
15        // Chargement des commentaires de test positifs et negatifs
16        // a partir des fichiers
17        Comments testComments = new Comments(Driver.DATASET_TEST_POS_PATH);
18        testComments.loadCommentFiles(Driver.POS_TAG);
19        testComments.setDataPath(Driver.DATASET_TEST_NEG_PATH);
20        testComments.loadCommentFiles(Driver.NEG_TAG);
21
22        // Creation de la base d'entrainement HBase
23        HBaseDB trainDB = new HBaseDB("train");
24        trainDB.createDataBase(trainComments);
25
26        // Creation de la base de test HBase
27        HBaseDB testDB = new HBaseDB("test");
28        testDB.createDataBase(testComments);
29    }
30 }
```

A8. Classe CassandraDriver

```
1 import java.io.IOException;
2
3 /**
4  * Created by fabrice on 9/24/15.
5  */
6 public class CassandraDriver extends Driver {
7     public static void main(String[] args) throws IOException {
8         // Chargement des commentaires d'entrainement positifs et negatifs
9         // a partir des fichiers
10        Comments trainComments = new Comments(Driver.DATASET_TRAIN_POS_PATH);
11        trainComments.loadCommentFiles(Driver.POS_TAG);
12        trainComments.setDataPath(Driver.DATASET_TRAIN_NEG_PATH);
13        trainComments.loadCommentFiles(Driver.NEG_TAG);
14
15        // Chargement des commentaires de test positifs et negatifs
16        // a partir des fichiers
17        Comments testComments = new Comments(Driver.DATASET_TEST_POS_PATH);
18        testComments.loadCommentFiles(Driver.POS_TAG);
19        testComments.setDataPath(Driver.DATASET_TEST_NEG_PATH);
20        testComments.loadCommentFiles(Driver.NEG_TAG);
21
22        // Creation de la base de donnees Cassandra
23        CassandraDB db = new CassandraDB("nfe204", "127.0.0.1");
24        db.connect();
25        db.createDataBase();
26        // Creation et remplissage de la table des donnees d'entrainement
27        db.createIMDBTable("train");
28        db.insertIMDBRecords(trainComments, "train");
29        // Creation et remplissage de la table des donnees de test
30        db.createIMDBTable("test");
31        db.insertIMDBRecords(testComments, "test");
32        db.disconnect();
33    }
34 }
```